UCAS

PostgreSQL Schema · Prisma ORM · Next.js 14 App Router

UniTime-Inspired Constraint Model · VPS Deployment · Electron-Ready

June 2026
Internal Architecture Document - Confidential

1. Database Design Philosophy

UCAS adopts a PostgreSQL relational database modelled after UniTime's proven timetabling schema (Apache 2.0 license), adapted and extended for South Asian university structures — specifically batch-prefix student cohorts (F22/S22), academic roadmaps, probation/deferment logic, and Excel-based bulk import. The schema is managed through **Prisma ORM**, giving type-safe database access from Next.js server actions and API routes.

2. What We Adopt from UniTime

UniTime's PostgreSQL schema (29,803 lines, Apache 2.0) was studied in detail. The table below maps UniTime constructs to our UCAS equivalents — what we copy, what we simplify, and what we add from scratch.

UniTime Table(s)	UCAS Table(s)	Action	Notes
preference_level	preference_level	COPY EXACTLY	5-level Required→Prohibited system used by all pref tables
time_pattern	time_pattern	COPY EXACTLY	Bitmask days + slot number. Makes clash detection O(1) integer comparison
date_pattern	date_pattern	SIMPLIFIED	We keep week-recurrence model; drop UniTime's complex 366-bit string
distribution_type + distribution_pref	distribution_type + distribution_pref	COPY EXACTLY	Constraint types (back-to-back, same-room, spread) between class pairs
solution + assignment	schedule_solution + assignment	COPY EXACTLY	One solution per permutation; assignments link class→time→room→instructor
instructional_offering → scheduling_subpart → class_	course → course_subpart → section	SIMPLIFIED	Same 3-level hierarchy but without cross-listing complexity
curriculum → curriculum_clasf → curriculum_course	academic_roadmap → roadmap_semester → roadmap_course	ADAPTED	UniTime's curriculum model reframed for our batch-prefix roadmap system
change_log	audit_log	COPY EXACTLY	obj_type, obj_uid, operation, old_value, new_value with hash chain
timetable_manager + roles	user + role + permission	SIMPLIFIED	Standard RBAC instead of UniTime's complex departmental manager hierarchy
sessions	semester	ADAPTED	Added type (Fall/Spring/Summer), batch_prefix linkage, isActive flag
departmental_instructor	instructor	SIMPLIFIED	Removed designator/career_acct; added track_type, max_credit_hours
student + course_demand + course_request	student + enrollment + course_demand	EXTENDED	Added batch_prefix, probation fields, deferment_plan linkage
building + room	building + room + lab	EXTENDED	Split lab as separate entity with specialization field

— (not in UniTime)	transcript + probation_record + deferment_plan	NEW	Full academic progress module — no equivalent in UniTime
— (not in UniTime)	academic_roadmap + roadmap_course	NEW	Batch-specific program roadmap with prerequisite chains
— (not in UniTime)	excel_import_log	NEW	Tracks every bulk import with row-level error reporting

COPY EXACTLY — taken from UniTime with minimal changes. **SIMPLIFIED** — UniTime concept kept but complexity reduced. **NEW** — no UniTime equivalent; built from scratch.

3. Complete PostgreSQL Schema

3.1 System & Access Control Tables

Table	Key Columns	Purpose
user	id UUID PK, email, password_hash, role_id FK, is_active, created_at	All application users across roles
role	id UUID PK, name (ADMIN/SCHEDULER/ACADEMIC/REPORTS), description	Seeded role definitions
permission	id UUID PK, module VARCHAR, action (CREATE/READ/UPDATE/DELETE)	Granular module-level permissions
role_permission	role_id FK, permission_id FK — composite PK	Many-to-many: roles to permissions
user_permission	user_id FK, permission_id FK, granted BOOL	Per-user override on top of role defaults
organization	id UUID PK, name, license_key, max_users, plan_type	Multi-org licensing (future multi-tenant)
audit_log	id UUID PK, user_id FK, action, entity, entity_id, old_val JSONB, new_val JSONB, hash, timestamp	Append-only tamper-evident log — from UniTime change_log
application_config	name VARCHAR PK, value TEXT, description — from UniTime	Key-value system settings (SMTP, grading, dates)
excel_import_log	id UUID PK, user_id FK, template_type, total_rows, success_rows, error_rows, errors JSONB, imported_at	Tracks every Excel bulk import operation

3.2 Scheduling Core Tables (UniTime-Inspired)

Table	Key Columns	Origin / Notes
preference_level	id INT PK, code CHAR(1), label VARCHAR, preference_value INT (-2=Required … +2=Prohibited)	COPIED FROM UNITIME — single source for all preference references
time_pattern	id UUID PK, semester_id FK, name, pattern_type, days_code INT (bitmask: Mon=1 Tue=2 Wed=4…), slots_per_mtg INT, slot INT (min/5 from midnight), break_time INT, min_per_mtg INT	COPIED FROM UNITIME — bitmask encoding enables O(1) clash check
date_pattern	id UUID PK, semester_id FK, name, pattern VARCHAR(52) (week bitmask), type, is_default	SIMPLIFIED from UniTime — 52-char string marks active teaching weeks
distribution_type	id UUID PK, reference VARCHAR, label, description, allowed_pref VARCHAR, sequencing_required BOOL, is_exam_pref BOOL	COPIED FROM UNITIME — seeded: SAME_ROOM, BACK_TO_BACK, SPREAD, MEET_TOGETHER, DIFFERENT_TIME

distribution_pref	id UUID PK, pref_level_id FK, dist_type_id FK, grouping INT	COPIED FROM UNITIME — constraint between two or more classes
distribution_object	id UUID PK, dist_pref_id FK, section_id FK, sequence_number INT	Links sections into a distribution constraint group
schedule_solution	id UUID PK, semester_id FK, label, status (DRAFT/PUBLISHED/ARCHIVED), fitness_score FLOAT, permutation_index INT, solver_params JSONB, generated_by FK, generated_at, published_at	One row per timetable permutation — equivalent to UniTime solution table
assignment	id UUID PK, solution_id FK, section_id FK, instructor_id FK, room_id FK (or lab_id), time_pattern_id FK, date_pattern_id FK, slot INT, days_code INT, is_locked BOOL, last_modified	Core scheduling unit — COPIED from UniTime assignment; is_locked prevents GA from moving it
clash_report	id UUID PK, solution_id FK, clash_type (ROOM/INSTRUCTOR/SECTION), assignment_a_id FK, assignment_b_id FK, description, auto_resolved BOOL	Auto-generated conflict log per solution
solver_parameter	id UUID PK, solution_id FK, name, value, description	Per-solution solver config (weights, timeout, iteration count)

3.3 Academic Entities

Table	Key Columns	Notes
semester	id UUID PK, name, type (FALL/SPRING/SUMMER), year INT, start_date, end_date, is_active BOOL, session_id (org-level)	Equivalent to UniTime sessions table
program	id UUID PK, name (BS/MS/PhD), code, total_credit_hours INT, min_gpa FLOAT, duration_semesters INT, dept_id FK	Degree program definition
department	id UUID PK, name, code, head_instructor_id FK, session_id FK	Academic department
subject_area	id UUID PK, dept_id FK, abbreviation, title — from UniTime	Subject grouping within department (e.g. CS, MATH)
course	id UUID PK, code VARCHAR UNIQUE, title, credit_hours INT, type (THEORY/LAB/BOTH), program_id FK, dept_id FK, subject_area_id FK, is_active	Master course record
course_subpart	id UUID PK, course_id FK, subpart_type (LECTURE/LAB/TUTORIAL), credit_hours INT, min_per_meeting INT, sessions_per_week INT	FROM UNITIME scheduling_subpart — separates lecture and lab components
course_offering	id UUID PK, course_id FK, semester_id FK, is_offered BOOL, max_enrollment INT, enrollment INT, external_uid	Course offered in a specific semester
section	id UUID PK, course_offering_id FK, course_subpart_id FK, label CHAR(1), capacity INT, enrollment INT, section_number INT	Individual section — equivalent to UniTime class_

course_outline	id UUID PK, course_id FK, version VARCHAR, content JSONB, effective_from DATE, effective_to DATE	Versioned course syllabus/outline
academic_roadmap	id UUID PK, program_id FK, batch_prefix VARCHAR (F22/S22), version INT, total_semesters INT, is_active	Program roadmap per batch cohort — maps to UniTime curriculum
roadmap_semester	id UUID PK, roadmap_id FK, semester_number INT (1-8), label VARCHAR	One row per semester in the roadmap — maps to UniTime curriculum_clasf
roadmap_course	id UUID PK, roadmap_semester_id FK, course_id FK, is_elective BOOL, prerequisites UUID[] (array of course_ids), ord INT	Course placement in roadmap — maps to UniTime curriculum_course

3.4 People — Instructors & Students

Table	Key Columns	Notes
instructor	id UUID PK, employee_id VARCHAR UNIQUE, fname, lname, email, designation_id FK, dept_id FK, track_type (TEACHING/RESEARCH), joining_date, is_active, external_uid	Simplified from UniTime departmental_instructor
instructor_designation	id UUID PK, title (Lecturer/AP/Associate/Professor), seniority_rank INT, max_credit_hours INT, exemption_policy JSONB	Designation with load limits
instructor_course_pref	id UUID PK, instructor_id FK, course_id FK, pref_level_id FK, is_expert BOOL	Course preference — references UniTime preference_level
time_pref	id UUID PK, owner_id (instructor or section), owner_type, time_pattern_id FK, pref_level_id FK	Time preference — references UniTime preference_level; COPIED from UniTime
room_pref	id UUID PK, owner_id, owner_type, room_id FK, pref_level_id FK	Room preference per instructor/section — COPIED from UniTime
building_pref	id UUID PK, owner_id, pref_level_id FK, building_id FK, distance_from INT	Building preference — COPIED from UniTime building_pref
instructor_availability	id UUID PK, instructor_id FK, semester_id FK, day_of_week INT, start_slot INT, end_slot INT (slot = min/5 from midnight)	Available time windows per semester
instructor_exemption	id UUID PK, instructor_id FK, type (SENIOR/LEAVE/RESEARCH/CUSTOM), start_date, end_date, details TEXT, approved_by FK, is_active	Teaching exemptions blocking assignment
instructor_load	id UUID PK, instructor_id FK, semester_id FK, assigned_credit_hours INT, max_credit_hours INT	Computed load; enforced during scheduling
student	id UUID PK, student_id VARCHAR UNIQUE (F22-BSCS-001), fname, lname, email, program_id FK, batch_prefix VARCHAR, enrollment_date, status (ACTIVE/PROBATION/DEFERRED/GRADUATED)	Batch prefix links to roadmap
enrollment	id UUID PK, student_id FK, section_id FK, semester_id FK, status (ENROLLED/DROPPED/COMPLETED/FAILED), is_repeat BOOL, enrolled_at	Student enrollment in a section
course_demand	id UUID PK, student_id FK, course_offering_id FK, priority INT, is_waitlist BOOL — from UniTime	Student's course request before sectioning
grade	id UUID PK, enrollment_id FK, marks_obtained FLOAT, total_marks FLOAT, grade VARCHAR, grade_points FLOAT, grading_type (RELATIVE/ABSOLUTE)	Grade per enrollment; grading type configurable per course/batch
transcript	id UUID PK, student_id FK, semester_id FK, semester_gpa FLOAT, cumulative_gpa FLOAT, credit_hours_attempted INT, credit_hours_earned INT	Computed per-semester academic summary

3.5 Infrastructure — Rooms & Labs

Table	Key Columns	Notes
building	id UUID PK, name, code, address, coordinate_x FLOAT, coordinate_y FLOAT, session_id FK, external_uid — from UniTime	Physical buildings; coordinates for distance constraints
room	id UUID PK, building_id FK, room_number, capacity INT, type (CLASSROOM/SEMINAR/LECTURE_HALL), features JSONB, external_uid	Teaching rooms — equivalent to UniTime room table
lab	id UUID PK, building_id FK, lab_number, capacity INT, specialization (CS/PHYSICS/CHEMISTRY/ELECTRONICS), features JSONB	Lab rooms — split from rooms for clarity
room_feature	id UUID PK, reference VARCHAR, label — from UniTime	Feature catalogue: PROJECTOR, WHITEBOARD, AC, etc.
room_feature_pref	id UUID PK, owner_id, owner_type, feature_id FK, pref_level_id FK — from UniTime	Feature preferences per instructor/section

3.6 Academic Progress & Probation Tables (No UniTime Equivalent)

Table	Key Columns	Purpose
probation_record	id UUID PK, student_id FK, semester_id FK, semester_gpa FLOAT, cumulative_gpa FLOAT, consecutive_low_count INT, status (WARNING/PROBATION_1/PROBATION_2), max_possible_gpa FLOAT, requires_deferment BOOL, notes TEXT	Tracks each GPA-check event; consecutive_low_count drives probation escalation
deferment_plan	id UUID PK, student_id FK, target_semester_id FK, repeat_courses UUID[] (array of enrollment_ids), status (DRAFT/PENDING_APPROVAL/APPROVED/ACTIVE/COMPLETED), created_by FK, approved_by FK, approved_at	Deferment/restart plan for second-probation students
deferment_timetable	id UUID PK, deferment_plan_id FK, assignment_id FK (selected schedule slot for each repeat course), generated_at	Per-course slot selections from current semester's published schedule
student_timetable	id UUID PK, student_id FK, solution_id FK, selected_assignments UUID[] (array of assignment_ids), generated_at, emailed_at	Clash-free personal timetable; can be regenerated anytime
grading_config	id UUID PK, course_id FK (nullable), program_id FK (nullable), batch_prefix VARCHAR (nullable), grading_type (RELATIVE/ABSOLUTE), grade_boundaries JSONB ({A:90,B:80...}), effective_from DATE	Configurable grading boundaries per course, program, or batch

4. Key Design Patterns

4.1 Preference Level System (Copied from UniTime)

A single **preference_level** table is the source of truth for all preference references across the entire application — room preferences, time preferences, building preferences, instructor preferences, and distribution preferences all foreign-key into it. This avoids scattered string enums and makes preference logic consistent everywhere.

preference_value	code	label	Used For
-2	R	Required	Room/time must be used — hard constraint at schedule level
-1	1	Strongly Preferred	Strong soft constraint — high penalty if violated
0	2	Preferred	Standard soft preference — moderate penalty
+1	3	Discouraged	Should avoid if alternatives exist
+2	P	Prohibited	Must not be assigned — equivalent to hard exclusion

4.2 Time Pattern Encoding (Copied from UniTime)

Instead of storing raw start/end datetime values, time slots are encoded as integers. This reduces clash detection to a simple bitwise AND operation between two slot bitmasks.

Field	Type	Example	Meaning
days_code	INT	21	Bitmask: Mon=1, Tue=2, Wed=4, Thu=8, Fri=16, Sat=32, Sun=64. 21 = Mon+Wed+Fri
slot	INT	96	Start time in units of 5 minutes from midnight. 96 × 5 = 480 min = 8:00 AM
slots_per_mtg	INT	12	Duration in 5-min units. 12 × 5 = 60 minutes per meeting
break_time	INT	2	Break within meeting in 5-min units. 2 × 5 = 10-min break
min_per_mtg	INT	50	Actual teaching minutes (slots_per_mtg × 5 – break_time × 5)

Clash detection: Two assignments clash if *(a.days_code & b.days_code) != 0* AND their time ranges overlap. With integer arithmetic, this runs in microseconds even for 1,000+ classes.

4.3 Solution & Assignment Structure (Copied from UniTime)

The scheduler generates K permutations. Each is a **schedule_solution** row. All class-slot-room-instructor assignments for that permutation are **assignment** rows linked to it. Publishing a solution simply sets its status to PUBLISHED. The GA optimizer scores each solution via a **fitness_score** float.

```
schedule_solution
  id, semester_id, label, status, fitness_score, permutation_index,
  solver_params JSONB, generated_by, generated_at, published_at
```

```
assignment   (one row per class in the solution)
  id, solution_id FK, section_id FK, instructor_id FK,
  room_id FK | lab_id FK,   time_pattern_id FK, date_pattern_id FK,
  slot INT, days_code INT, is_locked BOOL

clash_report   (auto-generated per solution)
  id, solution_id FK, clash_type, assignment_a_id, assignment_b_id,
  description, auto_resolved BOOL
```

4.4 Distribution Constraint Framework (Copied from UniTime)

Inter-class constraints (e.g. 'these two sections must share the same room' or 'this lecture and its lab must be on different days') are modelled as **distribution_pref** rows linking a **distribution_type** with a set of sections via **distribution_object** rows.

Distribution Type	Reference	Allowed Prefs	Meaning
Same Room	SAME_ROOM	Required / Strongly Preferred	All sections in the group must use the same room
Back to Back	BACK_TO_BACK	Preferred / Strongly Preferred	Sections should be in consecutive time slots
Spread	SPREAD	Required / Preferred	Sections spread across different days
Meet Together	MEET_TOGETHER	Required	Sections share the same time slot (combined class)
Different Time	DIFFERENT_TIME	Required / Prohibited	Sections must not overlap in time
Same Days	SAME_DAYS	Required / Preferred	Sections meet on the same days of the week
Same Start	SAME_START	Preferred	Sections start at the same time
Precedence	PRECEDENCE	Required	Section A must be scheduled before Section B

5. Next.js Platform Architecture

5.1 Tech Stack

Layer	Technology	Version	Purpose
Framework	Next.js App Router	14+	React server components, server actions, API routes
UI Components	shadcn/ui + Tailwind	latest	Accessible, composable component library
State	Zustand	4+	Global client state: active schedule, filters, selections
ORM	Prisma	5+	Type-safe PostgreSQL access; migration management
Database	PostgreSQL	15+	Primary data store on VPS
Auth	NextAuth.js v5	5	JWT sessions, credential provider, role middleware
Excel I/O	ExcelJS	4+	Template generation and bulk import parsing
PDF Export	Puppeteer	21+	Headless Chrome renders Next.js report pages to PDF
Email	Nodemailer	6+	SMTP email dispatch for schedules and timetables
Drag & Drop	dnd-kit	6+	Manual schedule board with real-time clash detection
Charts	Recharts	2+	Dashboard analytics, utilization heatmaps, CGPA trends
Validation	Zod	3+	Schema validation on all API inputs and Excel rows
Scheduling Algo	Custom TypeScript	—	CSP (backtracking + AC-3) + GA optimizer
Background Jobs	BullMQ + Redis	5+	Long-running schedule generation runs off the HTTP thread
File Storage	Local filesystem / S3	—	Uploaded Excel files; generated PDFs cache

5.2 Project Structure

```
ucas/
  src/
    app/                              Next.js App Router — pages & layouts
      (auth)/login/                 Login page (unauthenticated group)
      (dashboard)/                  Protected dashboard group
        admin/                        Admin pages: users, roles, audit, backup
        scheduler/                  Schedule generation, board, permutations
        academic/                     Progress, probation, deferment
        reports/                    Report catalogue and PDF viewer
      api/                              API routes (REST endpoints for async ops)
        schedule/generate/      POST — triggers BullMQ scheduling job
        import/[template]/      POST — Excel import per template type
        reports/[type]/           GET  — PDF generation endpoint
    components/                     Shared UI components
      schedule-board/             Weekly drag-drop timetable grid (dnd-kit)
```

```
        permutation-carousel/   Compare K generated schedules
        roadmap-timeline/         Student academic progress visualizer
    modules/                          Feature business logic
        scheduler/                  Constraint loader, solution builder
        progress/                    CGPA calc, probation engine, deferment planner
        importer/                    Excel parser + row validator per template
        reporter/                    PDF report builders (one file per report type)
    algorithm/                    Scheduling engine
        csp/                           CSP solver: variables, domains, backtracking, AC-3
        ga/                             GA optimizer: chromosome, fitness, crossover, repair
        clash/                        Clash detector using bitmask day comparison
        constraints/                One file per hard/soft constraint
    lib/                              Shared utilities
        prisma.ts                   Prisma client singleton
        auth.ts                      NextAuth config + middleware
        excel.ts                    ExcelJS helpers
        email.ts                    Nodemailer wrapper
        pdf.ts                       Puppeteer PDF wrapper
prisma/
    schema.prisma                 Full Prisma schema
    migrations/                    All DB migrations
    seed.ts                         Seed: preference_level, distribution_type, roles
public/templates/               Downloadable Excel import templates (16 files)
workers/                          BullMQ worker process for schedule generation
```

5.3 API Design

UCAS uses Next.js App Router with server actions for form submissions and mutations, and dedicated API routes for long-running or file-based operations. All API endpoints validate input with Zod and enforce role guards via NextAuth middleware.

Endpoint	Method	Role	Purpose
/api/import/[template]	POST	ADMIN / SCHEDULER	Upload Excel file; parse rows; return error report
/api/schedule/generate	POST	SCHEDULER	Queue BullMQ job; return jobId for polling
/api/schedule/[id]/status	GET	SCHEDULER	Poll generation job status and fitness score
/api/schedule/[id]/publish	PATCH	SCHEDULER	Publish a solution; archive other permutations
/api/reports/[type]	GET	ALL	Render report page via Puppeteer; return PDF binary
/api/email/teachers	POST	SCHEDULER	Bulk-email all teacher timetable PDFs
/api/email/student/[id]	POST	ACADEMIC	Email individual student timetable PDF

/api/progress/[studentId]	GET	ACADEMIC	Compute and return full academic progress summary
/api/probation/check	POST	ACADEMIC	Run probation engine for a semester; return records
/api/backup	POST	ADMIN	Dump PostgreSQL to encrypted ZIP; return download URL
/api/templates/[name]	GET	ALL	Download Excel import template file

5.4 Authentication & Role Guards

Role	Access Scope	Key Restrictions
ADMIN	All modules: Users, Roles, Audit, Backup, Config, Import, Schedule, Academic, Reports	Only role that can create/deactivate users and view audit log
SCHEDULER	Import, Schedule Generation, Manual Board, Room/Lab Management, Reports	Cannot view student grades, progress, or user management
ACADEMIC	Student Search, Progress Tracker, Probation, Deferment, Student Timetable, Enrollments, Reports	Cannot access scheduling board, user management, or system config
REPORTS	Read-only access to all published reports and PDF export only	No create/edit/delete operations; cannot trigger generation

NextAuth v5 middleware runs on every request to the dashboard group. It reads the JWT session's role claim and matches against a route-permission map. Server actions additionally re-check permissions server-side (never trust the client).

5.5 VPS Deployment

Component	Technology	Notes
Server	Ubuntu 22.04 LTS VPS (2 vCPU, 4 GB RAM minimum)	DigitalOcean / Hetzner / Vultr all work
Process Manager	PM2	Runs Next.js app + BullMQ worker; auto-restart on crash
Reverse Proxy	Nginx	SSL termination; proxies port 80/443 to Next.js on port 3000
SSL	Certbot (Let's Encrypt)	Free auto-renewing certificates
Database	PostgreSQL 15 (same VPS or managed DB service)	pg_dump for daily automated backups
Cache / Queue	Redis (same VPS)	BullMQ job queue for schedule generation; NextAuth session cache
File Storage	Local /uploads directory	Excel imports, generated PDFs; rsync to off-site for backup
Monitoring	Uptime Robot (free tier)	HTTP health-check; alerts on downtime
Updates	Git pull + prisma migrate deploy + pm2 restart	Zero-downtime rolling deploys via PM2 cluster mode

```
# Minimal Nginx config for UCAS
server {
     listen 443 ssl;
     server_name ucas.youruni.edu;
     ssl_certificate       /etc/letsencrypt/live/ucas.youruni.edu/fullchain.pem;
     ssl_certificate_key /etc/letsencrypt/live/ucas.youruni.edu/privkey.pem;
     location / {
           proxy_pass http://localhost:3000;
           proxy_http_version 1.1;
           proxy_set_header Upgrade $http_upgrade;
           proxy_set_header Connection 'upgrade';
           proxy_set_header Host $host;
           proxy_cache_bypass $http_upgrade;
     }
}
```

5.6 Electron Desktop Path (Future)

When a desktop build is required, the exact same Next.js codebase is wrapped in Electron with zero UI changes. The only additions are:

- electron/main.ts — Electron main process: creates BrowserWindow, loads localhost:3000
- electron/preload.ts — contextBridge for native OS dialogs (file open, print)
- Swap PostgreSQL connection string to a local PostgreSQL or SQLite (via Prisma datasource switch)
- electron-builder.config.json — packaging to .exe with NSIS installer
- Optional: machine fingerprint license check in main process before window loads

Total additional code for Electron wrapper: approximately 200–400 lines. All React components, Prisma schema, scheduling algorithm, and report templates remain unchanged.

6. Prisma Schema Snippet — Core Scheduling Tables

The following is a representative excerpt of the Prisma schema covering the core scheduling tables. The full schema (all 35+ models) will be committed to the repository as **prisma/schema.prisma**.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url         = env("DATABASE_URL")
  schemas   = ["timetable"]
}

model PreferenceLevel {
  id                     Int      @id @default(autoincrement())
  code                  String @unique @db.Char(1)
  label                 String
  preference_value Int
  // Relations
  time_prefs          TimePref[]
  room_prefs          RoomPref[]
  instructor_prefs InstructorCoursePref[]
  dist_prefs          DistributionPref[]
  @@schema("timetable")
}

model TimePattern {
  id                 String    @id @default(uuid())
  semester_id    String
  name               String
  pattern_type   String
  days_code       Int         // bitmask Mon=1 Tue=2 Wed=4 Thu=8 Fri=16 Sat=32
  slot               Int         // start time: minutes from midnight / 5
  slots_per_mtg Int         // duration in 5-min units
  break_time      Int         @default(0)
  min_per_mtg    Int
  semester         Semester @relation(fields: [semester_id], references: [id])
  assignments    Assignment[]
  @@schema("timetable")
}
```

```
model ScheduleSolution {
  id                        String          @id @default(uuid())
  semester_id          String
  label                   String
  status                  SolutionStatus @default(DRAFT)
  fitness_score       Float            @default(0)
  permutation_index Int
  solver_params       Json?
  generated_by         String
  generated_at         DateTime       @default(now())
  published_at         DateTime?
  semester               Semester       @relation(fields: [semester_id], references: [id])
  assignments          Assignment[]
  clash_reports       ClashReport[]
  student_timetables StudentTimetable[]
  @@schema("timetable")
}

enum SolutionStatus { DRAFT PUBLISHED ARCHIVED }

model Assignment {
  id                     String              @id @default(uuid())
  solution_id       String
  section_id         String
  instructor_id    String?
  room_id             String?
  lab_id               String?
  time_pattern_id String
  date_pattern_id String?
  slot                  Int
  days_code          Int
  is_locked          Boolean             @default(false)
  last_modified    DateTime            @updatedAt
  solution            ScheduleSolution @relation(fields: [solution_id], references: [id])
  section             Section              @relation(fields: [section_id], references: [id])
  time_pattern      TimePattern         @relation(fields: [time_pattern_id], references:
[id])
  clash_a             ClashReport[]      @relation("ClashA")
  clash_b             ClashReport[]      @relation("ClashB")
  @@schema("timetable")
}

model DistributionType {
  id                          String    @id @default(uuid())
```

```
  reference               String    @unique
  label                     String
  description             String?
  allowed_pref            String?   // e.g. "RP" = Required or Prohibited only
  sequencing_required Boolean   @default(false)
  distribution_prefs   DistributionPref[]
  @@schema("timetable")
}

model ProbationRecord {
  id                            String    @id @default(uuid())
  student_id                 String
  semester_id               String
  semester_gpa              Float
  cumulative_gpa           Float
  consecutive_low_count Int
  status                        ProbationStatus
  max_possible_gpa         Float?
  requires_deferment      Boolean   @default(false)
  notes                         String?
  student                      Student   @relation(fields: [student_id], references: [id])
  @@schema("timetable")
}

enum ProbationStatus { WARNING PROBATION_1 PROBATION_2 }
```

The full schema includes 35+ Prisma models covering all entities described in Section 3. Seed data will be provided for **PreferenceLevel** (5 rows), **DistributionType** (8 seeded types), and **Role** (4 roles) to bootstrap a fresh installation.